

Project Introduction

Our group ‘Git Happens’ built a CLI (Command Line Interface) application which utilises a Weather API to fetch raw and complex data from weather systems in JSON format. It then performs analysis and transforms the payload into readable reports and reusable data structures. The software provides a data pipeline from retrieval to multi-faceted storage.

Purpose:

- To fetch several data points from a weather API from user requests.
- Translate the raw data into a consistently usable format for the program.
- Utilise mathematical and statistical functions to convert into usable data points.
- Creates report files (JSON, CSV and TXT).
- Uses tools on report files that Save reports, Access reports, compare reports in CLI.

System Design & Technical Specifications

Technical Stack

Versions:	Python (3.12.3, 3.13.0, 3.13.9, 3.14.0, 3.14.2)
Core Libraries:	csv, json, os, requests, datetime, logging, statistics, typing, tkinter
APIs:	Visual Crossing, <i>Open Weather Map (legacy)</i>
Tools:	GitHub (Version Control), WhatsApp/Discord (Communication), Gemini (Doc String Generation)

Design Intentions

User experience:

- Launches a menu function for the user to navigate and pick their needs.
- Gives options for preference of reporting:
 - In-depth table of translated data.
 - Short analysis and overview of translated data.
- Allows the user to review the report in the terminal.
- Allows the user to then save the report to one of the existing report files. (Only Saves Simple)
- Designed to help anybody who needs to find weather data with customisation to allow for several use cases.

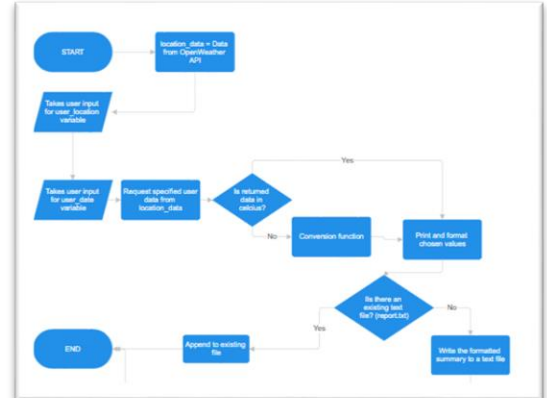
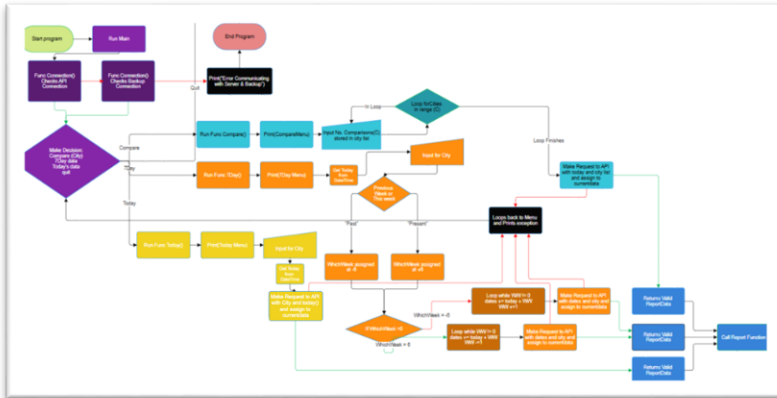
Developer Experience:

- Debugging and improvement of the program easier by utilising logs.
- Modular design and adheres to PEP 8 Standards.
- Accounts for edge cases, Error handling and has contingencies.

Repository Structure:

- “Final Submission” file containing finished code and report files.
- “Person Files” contains personal project development processes such as flowcharts and basic pseudocode.
- “README.md” contains the project brief and list of contributors.
- “Legacy Files” contains any development documents and code files used before the project was merged into one main.py file in the final submission folder.

Two Flowcharts were developed on by Caileb Cook and William Ellison to develop a vision of how we intended the program to run.



Check Appendix for image sources of flowcharts

Implementation Summary

Initial Strategy

In week 1 of the project, we met to discuss what our program was going to do, initialise up the GitHub repository and set up communication channels through WhatsApp and Discord. Caileb had designed a document that we had hoped to use as a framework for operations of the project which outlined division of labour and repository management (This can be found in CIS1702-CW2-Git-Happens/PersonalFiles).

Group Structure

The group of six was split into two sub-teams (Red and Blue). Each sub-team would have 3 weeks tasked with “API & Analysis” and “UI, Visualization & Reporting” Then rotated to the other task respectively to give everyone full stack exposure.

Review Cycle

We adopted a weekly cycle where everyone would upload code snippets to a shared staging folder to encourage code reviews before merging modules into the program.

Repository Design

The repository was pre-structured with specific directories for “documentation”, “api staging”, “ui staging” and “final submission”. This was done to separate non-production and production code. As the code developed and a single main.py file was established branches were created for subsequent additions to the code for future merge to prevent accidental coder override and protect previous additions.

Adaptation & Challenges

Whilst the initial plan built a strong framework and organised the work well, the reality of having other deadlines due and conflicting schedules required us to adapt. As the plan developed and we gained a better understanding of how GitHub can be implemented effectively by working from a single file in different branches which could then be merged, significantly improving the efficiency of the design flow.

Concerns

Having separation of workstreams through sub-teams lead to fragmented work and at first restricted team communication. As mentioned in our introduction members had produced independent drafts that became difficult to merge seamlessly because of the lack of use of branches and using separate files instead. Some of the file structure seemed redundant therefore was not utilised when implementing the merge into one file. After this no more merge issues or redundant code were present. There were some concerns for implementing the final user interface, “tkinter” however the implementation was made in the “UI_Interface” Branch then a new up to date branch “UI_Interface_Pre_Merge” was made to merge into to finalise the UI to new main branch components. The final merge was successfully.

Changes

Realizing that the schedule was impractical given the time constraints, we pivoted from the sub-team approach after drafts were made of a working program. After discussion William’s version became main.py, Caileb Implemented modules from his version with docstrings to main.py. We then met in discord and discussed what implementations were yet to be built and generated some ideas and then shared out the work each of us were happy to do. Main.py was pulled locally to add modules in staging, tested then merged to the main branch.

Legacy

Although a lot of the initial plan was dropped some of the repository structure remained such as individual notes and staging as they served as sandboxes to test features before deployment to main. All testing and legacy files have been saved in Legacy Files folder in the repository.

Improvement Phase

After we had moved over to one version of main.py that we would all work on via branches. The following improvements were made: The addition of a CSV comparison function, Clothing suggestion function, Date range option, UI “tkinter” integration, and finally testing/debugging.

API Class

Caileb’s method of using an ‘APIHandler’ class allowed the program to form a python dictionary upon connection with the API to avoid constantly re-calling the API. See below for iterations of Main, Importing Class to work within main’s constraints and the final iteration that is much cleaner, handles errors much better and stores local status codes for logging in a constant.

Transition from command line to GUI inputs and outputs

Before the implementation of the GUI, the code strictly used the command terminal to display data and ask for inputs. The transition to GUI was done carefully and stage by stage by using a series of branches to allow for continuous development of the GUI and main code separately by merging the GUI with an up-to-date branch and adding the GUI needed for the newly developed main branch features.

Code Functions Breakdown

save_report function

With the save function, a try and except block was utilised. Within the try block, code to save the report was completed and could be outputted in csv, json, or a txt file. Each style of report displays a timestamp, location, the date and all the data that was parsed from the API. The except block logs any error when it comes to saving the report to the errors log.

compare_csv function

The compare CSV function displays the current saved CSV data in “reporting.csv” to the user and asks for the to select 2 options. A table is then displayed which shows the information of both data sets with comparison data being calculated in a table form and as a text string on the conditions.

clothing_recommendation function

Whilst improving the main code, we decided to add a feature that suggests to the user what clothes they should use based on the current temperature. To do this, Corey used the already existing average temp variable which calculates an average temperature by finding the mean of values on the list of temperatures in the chosen area. A very simple but necessary function was created that outputted responses of clothes to wear into the simple report based on how hot or cold the location was.

Reporting functions

By switching to a modular design, the weather system was broken down into clear functions to allow us to create the most efficient program possible, that could be improved or edited with ease. These improvements are below:

- **Simple Report:**

For the simple report, the data outputted are the main weather condition, the average temperature (if the date selected is the current date), the highest and lowest temperature, and the chance of rain. The feature to suggest the type of clothes that should be worn is also there, so it does not intrude with the data returned in the detailed report.

- **Detailed Report:**

The detailed report returns an hourly table of temperature and conditions to give the user a better idea of how the whole day will look. Not incorporating the clothes recommendations here allows the data to be outputted in a clean and more readable way so the user can understand its purpose much easier.

improvements of run_reports menu

To improve the menu, it was decided to call each function individually within the if statements instead of writing unnecessary lines of code for it to output the same thing. This helped to improve the readability of the menu code.

Validation and Error Handling

Multiple versions and iterations of validation were used during development as the sanitation was improved over time. To handle errors and edge cases, input sanitation was used throughout the code to determine if the inputs that were given by the user would not interfere with the running of the program throughout the execution.

Try except blocks were implemented in places that could cause critical errors while running parts of the application (risky operations). Examples of this include when sending an API request and when data was saved to files. In addition to try except being used, opening files was performed by a ‘with open’ to automatically close the file editor. In certain exception cases, specific and descriptive error cases were used to help with debugging and error clarity.

Logging

Logging in the program was used for determining issues in the code, and also for feedback to the developers for where in the code was running. It has mainly been implemented in risky operations that were handled by a try except block, reporting a success, or a failure that has a descriptive reason to why the error occurred. The basic configuration of the logs that would be reported was defined to display time, severity level, and a specific message for each case. These logs are individually appended to the logging file AppErrors.log any time a log is used.

Code Menus before GUI implementation

Location Menu

```
Recent Cities: None
Recent Dates: None

Location (or 'Q' to quit): 230123

!!! Location not found. Please try again.
```

```
Recent Cities: None
Recent Dates: None

Location (or 'Q' to quit): Narnia

!!! Location not found. Please try again.
```

Upon inputting an invalid number or location the ‘validate_location’ function compares the user input with a .CSV which holds a list of cities, if a matching value isn’t found the program requests the user to try again.

Date Input Menu

```
Location (or 'Q' to quit): Liverpool
Date (YYYY-MM-DD) or Enter for today: Test!
OPTIONAL: End date (YYYY-MM-DD) Test!

!!! Invalid date format. Please use YYYY-MM-DD.
```

```
Location (or 'Q' to quit): Liverpool
Date (YYYY-MM-DD) or Enter for today: 2026-25-25
OPTIONAL: End date (YYYY-MM-DD)

!!! Data retrieval failed. Please check location/date and try again.
```

When entering a string or a non-valid date, the program has multiple failsafe’s, the ‘validate_date’ function and if/else statements in the main function to ensure the data exists before proceeding.

```
if data:
    run_reports(data, loc, date, date2)
else:
    print("\n!!! Data retrieval failed. Please check location/date and try again.\n")
    LOG.warning("Data retrieval failed after all API key attempts.")
```

```
if data:
    run_reports(data, loc, date, date2)
else:
    print("\n!!! Data retrieval failed. Please check location/date and try again.\n")
    LOG.warning("Data retrieval failed after all API key attempts.")
```

The program accounts for all other edge-cases such as:

- No inputted dates defaults to today's date,
- Inputting a second date alters the API link to receive data for a range of dates
- If a date range is requested but the first date isn't input (implying the current date), the first date value will be set using 'datetime.date.today()'

Report Menu:

```
Current View: LIVERPOOL | Today (Current)
1. Simple Summary
2. Detailed Hourly Breakdown
3. Save this report (CSV/JSON/TXT)
4. Back to Main Search
5. Compare CSV data

Enter selection (1-5): 6
Invalid choice, please try again.

Enter selection (1-5): Test
Invalid choice, please try again.
```

Report menu blocks all invalid inputs while successfully allowing the user to access all options between one and five. This is dealt with inside the 'run_report' function using an if/else statement to ensure only the values 1-5 are accepted.

Testing

The code was in constant testing as it was developed. However, the most significant testing took place when the final code was developed. Debugging and error handling could be performed on the whole code to make sure every aspect was accounted for. In addition, log files were implemented to keep track of errors values and test conditions.

What's Tested	Desired Outcome	Actual Outcome
Date input "2022/20/1" Invalid Format	Logs, Loops and reprompts	Logs, Loops and reprompts PASS
Location input "asdasd" Invalid Location	Logs, Loops and reprompts	Logs, Loops and reprompts PASS
Bad API Key	Logs, iterates to backup key, retry/break if end of index	Logs, iterates to backup key, retry/break if end of index PASS
Date Inputs wrong way around "2026-01-06" and "2026-01-05"	Valid Input and reporting functions follow	Invalid response and breaks FAIL Basic Error Handling added

Unused functions

Due to time limitations, the following functions were not implemented.

API key security:

This could have been implemented by utilising a .env file which would safely store the API key within secure environment variables. Due to time constraints, other functions were prioritised as the group would be the only people using the API key anyways.

Being able to save hourly data:

This could have been implemented alongside the csv comparison, however time constraints and the need to finalise the GUI took priority over this feature.

UserQuery Class

Cached user queries from the time they used the application and can call previous reports until the program is closed.

Breakdown of Contribution

Ben – Implement the GUI he had been working on to be functional within `##WeatherApp.py`. The later development of the GUI was done alongside the `main.py` in separate branches within the repository which were later merged for a final GUI system.

Caileb – Created `caileb_staging.py`, Created API & UserQuery Classes, Developed Reporting to include Simple and Detailed report to Terminal, Developed `save_report` to save to json and txt also, significantly improved error handling with logs and core functions. Improved overall readability, efficiency, scalability and cleanliness of the program. Spent significant time on the group report. Organising some of the project's DevOps and delegation of labour.

Corey: Implemented function to suggest to the user what type of clothes they should wear based on the current temperature to the final submission code. Built `WEATHERAPI_CM.py` which included functions that are very alike to the core functions in the final submission.

Daniel: Implemented input sanitation to report menu and other areas with terminal inputs. Implemented a logging function that appropriately logs info, warnings, and errors. Implemented a feature that helps save the logs to a log file. Assisted with troubleshooting and debugging the code during development.

Isaac: Implemented function that compares CSV weather reports saved, I also added menu override prevention, so the while true main menu does not immediately display after making a report or extended hourly display. The CSV function originally in branch “Staging-for-File-Save-and-analysis” had to be altered during the implementation using a single file. Furthermore, the final iteration was also adjusted during the implementation of the GUI, making some of the code redundant but mainly just restructuring was needed.

William – Created `##Weather_app.py` draft, implemented a function that allows reports weather within a range of dates. Updated logic in ‘`show_simple_report`’ function, API handler class as well as the main function. Writing of command line testing and Group reflection within the group report.

Group Reflection

Overall, as a group we are happy with our final product, while we did come scarily close to the submission deadline and faced various challenges, we have implemented a variety of features while keeping our code bug-free and accounting for any edge-cases discovered.

During our time working together we have realised the importance of clear communication and source control. Diverting from original plans and lack of communication led to pre-

deadline crunch. At the start of the project, we all had limited experience with Git, leading to multiple fragmented versions of code and multiple instances of updating code to older versions, sometimes making the updates dysfunctional. Since then, we have all become more conditioned to programming in a group setting by learning how to properly use Git, leaving comments to explain what has been updated and letting the group know what you are working on to avoid duplicate modules. Even though we have procured an operational and extensive final product, we can all comfortably say that when we are next faced with a group project, we will be able to perform much more efficiently due to the lessons learned in ‘Git Happens’.

Individual Reflection

Ben: I believe that the GUI that was implemented was done very well, but it took too long to fully implement. I also believe I could have communicated with the team a little better to make troubleshooting the GUI much easier. Furthermore, because of the complexity of the GUI system, multiple branches were needed to stage the integration through controlled merges and manual merge conflict solving.

Caileb: This project has been fun, reviewing drafts of main.py taught me how I can now make my code cleaner and easier to understand. Working with library limitations developed a deeper knowledge about problem solving. I was able to get industry feedback, it was invaluable and highlighted the importance of docstrings to provide module context. Next time I plan to engage more with my team to help develop the DevOps plan more effectively.

Corey: Individually, I believe that for the most part, my code went quite well, due to not having many errors. I do think that learning how to fully utilise functions used in the main program earlier on would have helped massively to fully refining my code. I could have improved by utilising git more to explain what I was changing in more depth to allow my team to understand my changes easier.

Daniel: Overall, I think that working on this project was a valuable experience for me. The development allowed me to practice my python skills while developing a really cool tool that actually serves a purpose. I enjoyed adding logging the most so that the code could be tested fully. I think I could have started working on the project earlier, but as soon as I started coding I had fun trying to find the best solution to any problems I encountered. My weakest area was probably in integrating my development of the code alongside github’s version control.

Isaac: In reflection I could have done more pseudocode planning for the implementation of the CSV comparison. The CSV also uses multiple nested loops, and the function does not return the data vales so it cannot be used outside of the function, an issue which was realised when implementing the GUI. This project has been challenging and has improved my teamworking and python skills.

William: Personally, I enjoyed the process of developing the weather program, I learned multiple skills along the way such as API usage and classes within Python while becoming much more accustomed with Git. For the future I’d like to get more experience with elements I wasn’t involved in for this project such as logging systems and interacting with JSONs, CSV and text files. While

this project has been stressful at times, I believe the challenges faced increased my programming knowledge and group programming abilities.

Conclusion

To conclude, our project was quite successful. We created a working program that makes use of a GUI to output our data in a much cleaner way. We have demonstrated our understanding of the process for creating our program; from establishing a connection to a public API to parsing complex data to use to our advantage. The team GitHub collaboration only became more successful when we managed to properly implement the branch and merge system. Overall, we are happy with the project, and although in future we would approach it differently as we now have a much better understanding how to effectively collaborate via GitHub.

Appendix:

Group Members:

Benjamin Elliott, Caileb Cook, Corey Mairs, Daniel Hassett, Isaac Ritchie, William Ellison

Repository Link:

<https://github.com/Isaac-D-Ritchie/CIS1702-CW2-Git-Happens>

Flowcharts:

https://github.com/Isaac-D-Ritchie/CIS1702-CW2-Git-Happens/blob/main/Personal%20Files/WilliamE_26205432_Flowchart.png
https://github.com/Isaac-D-Ritchie/CIS1702-CW2-Git-Happens/blob/main/Personal%20Files/CailebC_26553767.md

Iterations of API Handling:

```
api_key = '247BUSAULFLBGWM7NVZ49M4B'

def main():
    while True:
        print("=== Weather Data ===\n")
        user_location = input("Enter location:\n")
        user_date = input("Enter date in a YYYY-MM-DD format (Leave blank for current weather)")
        user_choice = input(f"Location:{user_location}\n Date:{user_date}\n Continue? (Y/N)")
        user_choice = user_choice.lower()
        if user_date == "":
            user_date = None #Allows user_date to be passed into the function even with no value
        if user_date != "":
            user_date = user_date + "/" #Adds a slash for the API link, slash isnt needed if date isn't input
        if user_choice == "y":
            location_data = load_location_data(user_location, user_date, api_key)
            weather_values(location_data, user_location, user_date)

def load_location_data(user_location, user_date, api_key):
    location_data = requests.get(f'https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/timeline/{user_location}/{user_date}?key={api_key}')
    return location_data

def fahrenheit_to_celcius(fahrenheit): #Visual Crossing API provides fahrenheit values
    celcius = (fahrenheit - 32) / 1.8
    return celcius

def weather_values(location_data, user_location, user_date):
    temp_current = fahrenheit_to_celcius(location_data.json()['days'][0]['temp']) #Parsing data from the API
    temp_max = fahrenheit_to_celcius(location_data.json()['days'][0]['tempmax'])
    temp_min = fahrenheit_to_celcius(location_data.json()['days'][0]['tempmin'])
```

```
class APIHandler:
    def __init__(self, api_key):
        #API Basic info
        self.url = ('https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/timeline/')
        self.keys = [api_key, 'BACKUPAPIKEY']

        #Connect Function
    def connect(self, location, date1, date2=None, index = 0):
        if index >= len(self.keys):
            print("CRITICAL ERROR: ALL API CONNECTION DATA FAILED, EXITING") #!! LOG
            return None
        #Handles URL Date, decides between using just one date or two for range data
        date_path = f"{date1}{date2}" if date2 else f"{date1}"
        full_url = f'{self.url}{location}/{date_path}?key={self.keys[index]}'

        if index == 0:
            print(f"Attempt {index + 1}: Connecting to {self.url} (Primary Key)") #!! LOG
        else:
            print(f"Attempt {index + 1}: Connecting to {self.url} (Backup Key)") #!! LOG

        try:
            with requests.get(full_url, timeout=10) as response:
                if response.status_code == 200:
                    print("Connection successful!") #!! LOG
                    return response.json()

            #Recursive step given failed connection
        except Exception as e:
            print(f"Error: {e}\nAttempt {index+1} failed, Trying next option...") #!! LOG
            return self.connect(index+1)

class APIHandler:
    """
    """

    def __init__(self, location: str, date: str = "", date2: str = ""):
        """Initializes the APIHandler with user query parameters."""
        LOG.info('API Handler Initialized.')

        self.location = location
        self.date = date
        self.date2 = date2
        self.api_data = {
            'url': 'https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/timeline/',
            'keys': ['247BUSAULFLBGWM7NVZ49M4B', 'U4QN48S3UU3XJ2C3LAK6TC8MM']
        }

    def connect(self) -> Optional[Dict[str, Any]] | None: #Returns JSON
        """
        Attempt to fetch weather data by iterating through available API keys.

        Returns
        -----
        dict or None
            A dictionary containing the parsed JSON response if successful,
            otherwise None.
        """
        url = self.api_data['url']
        for key in self.api_data['keys']:
            if self.date == "":
                self.date = datetime.date.today()

            date_path = f"/{self.date}" if self.date else ""
            if self.date2:
                date_path += f"/{self.date2}"
            full_url = f'{url}{self.location}{date_path}?key={key}&include=hours'

            try:
                LOG.info(f"Connecting to API for location {self.location}...")
                with requests.get(full_url, timeout=10) as response:
                    response.raise_for_status()
                    LOG.info(STATUS_CODES.get(str(response.status_code), "| Status Code Received |"))

                    if response.status_code == 200:
                        LOG.info(f"Success with Key ending in ..{key[-4:]}")
                        return response.json()

            except (HTTPError, JSONDecodeError, Exception) as err:
                LOG.warning(f"Attempt with key ..{key[-4:]} failed: {err}")

        LOG.critical("All API keys failed. Returning to Main.")
        return None
```

```
STATUS_CODES = {  
    #success  
    '200': '| JSON Recieved | Data Present and Formatted Correctly |',  
    '204': '| JSON Recieved | Data Not Present, Formatted Correctly |',  
    #client error  
    '400': '| Bad Request | Improper Query or Bad JSON Recieved |',  
    '401': '| Bad API Key | Invalid or None API Key used |',  
    '403': '| Forbidden | Fetch request is valid but no permission is given |',  
    '404': '| Not Found | Requested resource does not exist |',  
    '429': '| Rate-Limit Exceeded| Too Many Requests, Try again later! |',  
    #server error  
    '500': '| Internal Server Error | Unexpected Error with Server |',  
    '502': '| Service Unavailable | Try again Later, Servers may be under maintenance |'  
}
```